```
transformer_X = X.clone()
transformer_y = y.clone()
```

```
import math

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super(PositionalEncoding, self).__init__()
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        self.register_buffer('pe', pe)

    def forward(self, x):
        x = x + self.pe[:x.size(1), :]
        return x
```

```
class TransformerGenerator(nn.Module):
    def __init__(self, vocab_size, d_model, nhead, num_layers):
        super(TransformerGenerator, self).__init__()
        self.embedding = nn.Embedding(vocab_size, d_model)
        self.pos_encoder = PositionalEncoding(d_model)
        encoder_layers = nn.TransformerEncoderLayer(d_model, nhead, batch_first=True)
        self.transformer_encoder = nn.TransformerEncoder(encoder_layers, num_layers)
        self.fc = nn.Linear(d_model, vocab_size)

    def forward(self, x):
        x = self.embedding(x)
        x = self.pos_encoder(x)
        x = self.transformer_encoder(x)
        out = self.fc(x[:, -1, :])
        return out
```

```
transformer_model = TransformerGenerator(vocab_size=vocab_size, d_model=64, nhead=4, num_layers=2)
transformer_optimizer = torch.optim.Adam(transformer_model.parameters(), lr=0.005)
transformer_criterion = nn.CrossEntropyLoss()

transformer_epochs = 100
for epoch in range(transformer_epochs):
    transformer_optimizer.zero_grad()
    outputs = transformer_model(transformer_X)
    loss = transformer_criterion(outputs, transformer_y)
    loss.backward()
    transformer_optimizer.step()
```

```
seed_transformer = "sequence models process information"
print(generate_sequence(transformer_model, seed_transformer, 5))
```

```
sequence models process information step by step by step
```